

BCSE-008: Computational and Problem Solving using 'C'

Department of Computer Science & Engineering • MM (Deemed to be University), Mullana (Ambala)

UNIT I (25%)

Fundamentals, Logic Types, Complexity, Debugging, 4-Stage Pipeline, Memory Layout.

UNIT II (25%)

Tokens, Operators, 9-Tier Precedence, Typecasting, Control Flow, Looping & Jumps.

UNIT III (25%)

Arrays (Row-Major), Search/Sort, Strings, 12-pt Storage Classes, Modularity & Recursion.

UNIT IV (25%)

Pointers (Near/Far/Huge), Struct vs Union, Dynamic Memory (DMA), File I/O Streams.

UNIT I: PROGRAMMING FUNDAMENTALS, ARCHITECTURE & PROBLEM SOLVING

Computational Problems, Deductive/Inductive Logic, Algorithmic Complexity, Debugging Strategies, 4-Stage Compilation Pipeline & Process Memory Layout

1.1 Computational Problem Classification & Logic Types

Syllabus Section 1

Problem-solving in computational science is divided into **Decision Problems** (yes/no outcome), **Counting Problems** (determining quantity of valid configurations), and **Optimization Problems** (finding the globally optimal candidate among alternatives). Program logic is established using two primary paradigms:

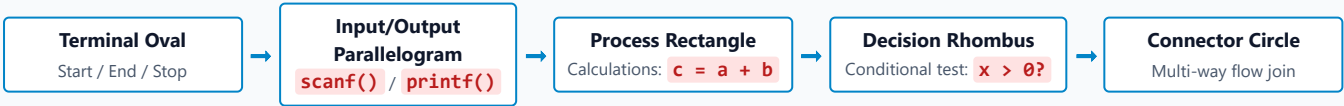
Logic Paradigm	Direction & Process	Validation Guarantee	Engineering Example in C
Deductive Logic	Top-Down: Starts with universal axioms and applies formal rules of inference to derive specific true conclusions.	Guaranteed Certainty: If premises are true, conclusions are inviolably correct.	Proving loop termination via loop invariants: If <code>i=0</code> and incremented by 1, <code>i < N</code> terminates in exactly <code>N</code> steps.
Inductive Logic	Bottom-Up: Observes specific sample behaviors and infers general algorithmic heuristics or predictive patterns.	Probabilistic / Plausible: True for tested inputs, but unverified for edge cases.	Black-box unit testing: Passing 10 positive test vectors does not mathematically guarantee absence of memory leaks.

1.2 Algorithmic Design, Flowcharts & Stepwise Refinement

Top-Down Methodology

Stepwise Refinement (Top-Down Decomposition): Complex software is iteratively decomposed into smaller, self-contained sub-modules until each subroutine maps directly to an elementary operation. Standard ANSI Flowcharting symbols govern structural design:

FIGURE 1.1: ANSI STANDARD FLOWCHART SYMBOL HIERARCHY



1.3 Debugging Strategies, Testing & Error Classification

Quality Assurance

Software defects fall into four distinct tiers: (1) **Syntax Errors** (grammatical violations detected at compile-time, e.g., missing semicolons), (2) **Linker Errors** (unresolved function declarations, e.g., calling undefined functions), (3) **Runtime Errors** (abrupt OS aborts such as division by zero or SIGSEGV memory faults), and (4) **Semantic/Logic Errors** (syntactically valid programs that yield invalid outputs, e.g., off-by-one errors).

BLACK-BOX VS WHITE-BOX TESTING

- Black-Box Testing:** Assesses functional requirements purely against inputs and expected outputs without visibility into source code. Focuses on boundary-value analysis and equivalence partitioning.
- White-Box Testing:** Inspects internal code structures, guaranteeing statement coverage, branch/decision coverage, and loop boundary conditions.

DRY RUN EXECUTION TRACING TABLE EXAMPLE

Step	Line	Variable <code>i</code>	Variable <code>sum</code>	Condition <code>i <= 3</code>
1	1	1	0	1 <= 3 (True)
2	2	1	1 (0 + 1)	-
3	1	2	1	2 <= 3 (True)
4	2	2	3 (1 + 2)	-
5	1	3	3	3 <= 3 (True)
6	2	3	6 (3 + 3)	-
7	1	4	6	4 <= 3 (False → Abort)

1.4 Algorithmic Complexity: Big-O Asymptotic Notation

Time & Space Analysis

Notation	Name	Operations for N=1000	Typical C Algorithm Example
<code>O(1)</code>	Constant Time	1 operation	Array index lookup <code>a[i]</code> , pointer dereference <code>*p</code> .
<code>O(log N)</code>	Logarithmic Time	~10 operations	Binary search in sorted array (halves problem space).
<code>O(N)</code>	Linear Time	1,000 operations	Linear search, single loop array traversal.
<code>O(N log N)</code>	Linearithmic Time	~10,000 operations	Merge Sort, Quick Sort (average case divide & conquer).
<code>O(N²)</code>	Quadratic Time	1,000,000 operations	Bubble Sort, Selection Sort, nested matrix iterations.

1.5 The 4-Stage Compilation Pipeline & Process Memory Layout

System Architecture

FIGURE 1.2: COMPLETE 4-STAGE C COMPILATION FLOW

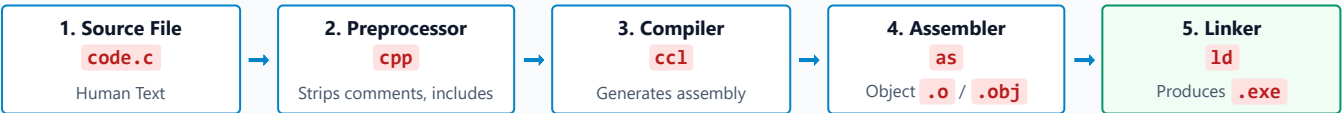
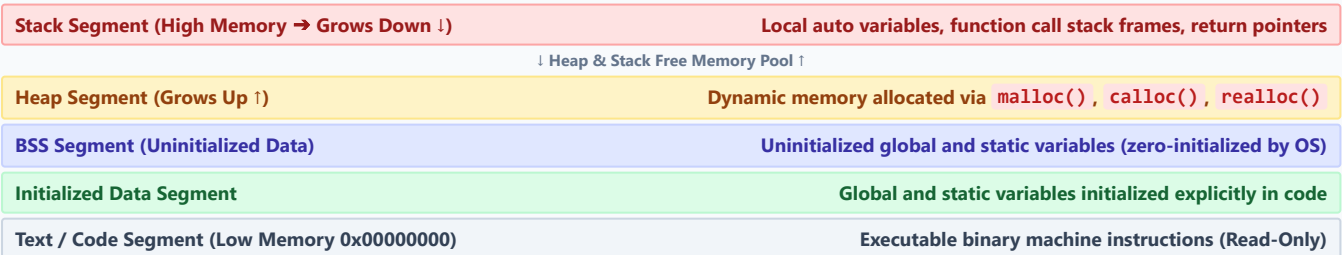


FIGURE 1.3: C PROCESS MEMORY ARCHITECTURE IN VIRTUAL RAM



UNIT II: CORE PROGRAMMING CONCEPTS, OPERATORS & CONTROL STRUCTURES

C Tokens, Primitive Types, 9-Tier Precedence & Associativity, Type Conversion, Header Files Directory, Decision Making & Looping Lifecycles

2.1 C Tokens, Primitive Data Types & Format Specifiers

Lexical Foundations

Tokens are the smallest individual units recognized by the C compiler, comprising **Keywords** (32 reserved identifiers like `int`, `volatile`, `sizeof`), **Identifiers** (names created by programmer adhering to regex `[a-zA-Z_][a-zA-Z0-9_]*`), **Constants**, **String Literals**, and **Operators**.

Data Type	Size in RAM	Format Specifier	Range / Precision	Memory Encoding
char	1 Byte (8 bits)	%c	-128 to 127 (or 0 to 255 unsigned)	ASCII / 2's Complement
int	4 Bytes (32 bits)	%d, %i	-2,147,483,648 to 2,147,483,647	Signed 2's Complement
unsigned int	4 Bytes	%u	0 to 4,294,967,295	Pure binary magnitude
float	4 Bytes	%f	3.4E-38 to 3.4E+38 (6 decimal digits)	IEEE 754 Single Precision
double	8 Bytes	%lf	1.7E-308 to 1.7E+308 (15 decimal digits)	IEEE 754 Double Precision
void* / Pointer	4 B (32-bit) / 8 B (64-bit)	%p	Full virtual address space	Raw unsigned hexadecimal address

2.2 Complete 9-Tier Operator Precedence & Associativity Hierarchy

Grammar & Evaluation

Tier	Category	Operators	Associativity	Evaluation Rules & Pitfalls
1 (Highest)	Postfix / Subscript	() [] -> . ++ --	Left-to-Right	Subscripts and struct dereferencing resolve before any arithmetic.
2	Unary	+ - ! ~ ++ -- (type) * & sizeof	Right-to-Left	Prefix ++x, address-of &, dereference * evaluate right-to-left.
3	Multiplicative	* / %	Left-to-Right	Modulus % requires integers on both operands (compiler error with floats).
4	Additive	+ -	Left-to-Right	Standard arithmetic addition and subtraction.
5	Relational	< <= > >=	Left-to-Right	Yields integer boolean 1 (True) or 0 (False).
6	Equality	== !=	Left-to-Right	Lower precedence than relational: a < b == c evaluates as (a < b) == c.
7	Logical	&& (AND) then (OR)	Left-to-Right	Short-Circuit: In A && B, if A is false, B is skipped! In A B, if A is true, B is skipped!
8	Ternary Conditional	? :	Right-to-Left	condition ? expr1 : expr2. Nested ternaries associate right-to-left.
9 (Lowest)	Assignment & Comma	= += -= *= /= %= ,	R-to-L (=) L-to-R (,)	Comma operator evaluates expressions left-to-right and yields rightmost value.

2.3 Type Conversion: Implicit Promotion vs Explicit Typcasting

Type Safety

Implicit Conversion (Type Promotion): Automatically performed by compiler following the rank order: `char` → `int` → `unsigned` → `long` → `float` → `double` → `long double`. Truncation danger: `int a=5, b=2; float c = a/b;` stores `2.000000` because integer division truncates to `2` prior to assignment.

Explicit Typcasting: Programmer forces conversion syntax `(target.type)expression`: `float c = (float)a / b;` promotes `a` to float first, resulting in true float division `2.500000`.

SWITCH-CASE FALL-THROUGH MECHANICS

In C `switch(expr)`, `expr` must resolve to an integer/char constant. Omitting `break;` induces **Fall-Through** where control spills sequentially into subsequent cases regardless of matched condition:

```
switch(grade) {
    case 'A': printf("Excellent "); // No break!
    case 'B': printf("Good ");      // Executes if 'A' or 'B'!
                break;
    default: printf("Average");
}
```

JUMPING STATEMENTS: BREAK, CONTINUE & GOTO

Statement	Behavior	Loop State Impact
<code>break;</code>	Instantly terminates enclosing loop or switch block.	Execution jumps past loop closing <code>}</code> .
<code>continue;</code>	Bypasses remaining statements of current pass.	Jumps directly to loop update <code>i++</code> step.
<code>goto label;</code>	Unconditional jump directly to <code>label:</code> .	Violates structured hierarchy; leads to spaghetti code.

UNIT III: DATA HANDLING USING ARRAYS, STRINGS, FUNCTIONS & MODULARITY

1D & 2D Array Mapping (Row-Major Formula), Searching & Sorting Algorithms, 12-Point Storage Classes Matrix, Call by Value vs Reference & Recursion

3.1 1D and 2D Arrays: Memory Layout & Row-Major Addressing Formula

Arrays are homogeneous, contiguous collections of memory elements. In a 1D array of type `T` with base address `B`, element `A[i]` resides at: $Addr(A[i]) = B + i * sizeof(T)$.

MATHEMATICAL DERIVATION: ROW-MAJOR ORDER 2D ARRAY ADDRESSING

In C, multi-dimensional arrays are stored in **Row-Major Order** (entire row 0 is stored consecutively, followed by row 1, etc.). For a matrix declared as `A[R][C]` where `R` is row count, `C` is column count, and element size is `S` bytes:

Address(A[i][j]) = BaseAddress + [(i × C) + j] × Size,Element

Numerical Proof: For `int A[4][5]` starting at Base=2000 with `sizeof(int)=4`, element `A[2][3]` skips 2 full rows (each row contains 5 elements) plus 3 columns in the current row: $Offset = (2 \times 5 + 3) \times 4 = 13 \times 4 = 52$ bytes. Therefore, $Address(A[2][3]) = 2000 + 52 = 2052$.

3.2 Searching and Sorting: Linear Search, Binary Search & Bubble Sort

Algorithm	Pre-conditions	Best Time	Worst Time	Space	Algorithmic Mechanism
Linear Search	None (Unordered array)	$O(1)$	$O(N)$	$O(1)$	Sequentially compares search key against indices <code>0</code> through <code>N-1</code> .
Binary Search	Array must be strictly sorted!	$O(1)$	$O(\log N)$	$O(1)$	Computes $mid = low + (high-low)/2$. If $key < a[mid]$, search left; else search right.
Bubble Sort	None	$O(N)$ (optimized)	$O(N^2)$	$O(1)$	Repeatedly compares adjacent elements $a[j] > a[j+1]$ and swaps them, sinking max to end.

FIGURE 3.1: BUBBLE SORT STEP-BY-STEP PASS TRACE ON [5, 1, 4, 2]

Pass 1: [5, 1, 4, 2] → Swap(5,1) → [1, 5, 4, 2] → Swap(5,4) → [1, 4, 5, 2] → Swap(5,2) → [1, 4, 2, 5] (5 sunk to end)
Pass 2: [1, 4, 2, 5] → No Swap(1,4) → Swap(4,2) → [1, 2, 4, 5] (4 sunk to position)
Pass 3: [1, 2, 4, 5] → No swaps detected → Array Fully Sorted in $O(N)$ optimized passes!

3.3 Exhaustive 12-Point Master Storage Classes Matrix

MMDU Mandatory Reference

Storage Class	Keyword	Storage Location	Initial Default	Scope / Visibility	Lifetime	Linkage Type	Key Exam Behavior
Automatic	auto	Stack Memory	Garbage value	Local to enclosing block	Function invocation	None (internal)	Default for all local variables. Automatically destroyed upon function return.
Register	register	CPU Microprocessor Register	Garbage value	Local to enclosing block	Function invocation	None	Optimization hint for fast loop counters. The address-of operator &var is ILLEGAL!
Static	static	Data Segment (BSS / Data)	Zero (0 / '')	Local to block or file	Entire program execution	Internal linkage	Retains value across repeated function calls. Initialized exactly once at compile-time.
External	extern	Data Segment	Zero (0 / '')	Global across all files	Entire program execution	External linkage	References global variable allocated in another translation unit. Allocates no new memory.

3.4 Parameter Passing: Call by Value vs Reference & Recursion Stack Frames

Modular Architecture

CALL BY VALUE VS CALL BY REFERENCE

Feature	Call by Value	Call by Reference
Arguments	Copy of variable passed	Memory address (&x) passed
Caller Mutation	Changes do NOT affect caller	Dereference *p mutates caller
Syntax	void f(int a)	void f(int *a)

RECURSION STACK FRAME ACTIVATION TRACE FOR 3!

Call fact(3) → Pushes Frame 3 [N=3, waits 3 * fact(2)]
Call fact(2) → Pushes Frame 2 [N=2, waits 2 * fact(1)]
Call fact(1) → Pushes Frame 1 [N=1, Base hit: returns 1]
Unwind → Frame 2 computes 2 * 1 = 2 → Frame 3 computes 3 * 2 = 6.

UNIT IV: ADVANCED DATA HANDLING, POINTERS, STRUCTURES, DMA & FILE I/O

Pointer Taxonomy (Near/Far/Huge, Function Pointers), Struct vs Union Architecture, Dynamic Memory Allocation (DMA), File Stream Modes

4.1 Comprehensive Pointer Taxonomy & Real-Mode 16-Bit Architecture

Memory Addressing

Pointers store memory addresses. Beyond basic types, the MMDU syllabus mandates mastery over specialized pointer classifications:

Pointer Classification	Definition & C Syntax	Memory Behavior & Hazard	Correction / Best Practice
Wild Pointer	int *ptr; (Uninitialized)	Points to arbitrary memory; dereferencing causes catastrophic crashes or memory corruption.	Always initialize immediately: int *ptr = NULL;
Dangling Pointer	free(ptr); (Target deleted)	Points to deallocated heap storage that may be reassigned to other subsystems.	Immediately nullify after freeing: free(ptr); ptr = NULL;
Void Pointer (Generic)	void *vptr = &x;	Points to any data type without type binding. Cannot be directly dereferenced.	Must explicitly typecast before dereferencing: *(int*)vptr.
Function Pointer	int (*fp)(int, int) = ∑	Holds entry-point address of executable code in text segment. Enables callbacks.	Invoked via int res = fp(10, 20);.
Near Pointer	16-bit (2 Bytes) Offset address	Limited strictly to 64 KB of data within current default Segment Register (DS/CS).	Fastest arithmetic in 16-bit real-mode architecture.
Far Pointer	32-bit (4 Bytes): 16-bit Segment : 16-bit Offset	Can address full 1 MB 8086 RAM. Address comparison fails if pointers are unnormalized!	Requires segment normalization before relational comparisons.
Huge Pointer	32-bit Normalized Address	Automatically normalizes segment:offset on every pointer arithmetic increment.	Reliable pointer arithmetic and comparisons across 64 KB boundaries.

4.2 Structures, Unions & Memory Alignment Architecture

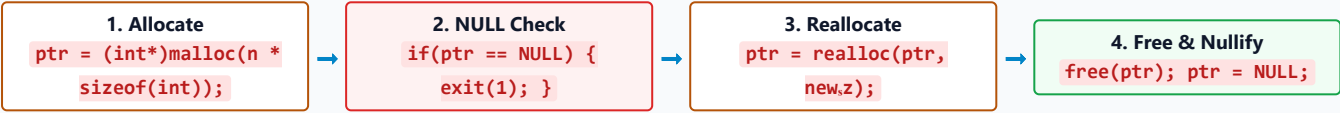
Heterogeneous Records

Comparative Dimension	Structure (struct)	Union (union)
Keyword & Declaration	<code>struct Record { int id; char grade; double score; };</code>	<code>union Record { int id; char grade; double score; };</code>
Memory Allocation	Allocates sum of all member sizes + structure padding bytes.	Allocates space equal only to the size of its largest member.
Starting Address	Each member has its own unique, sequential offset address.	All members share the exact same starting memory address.
Active Members	All members can hold independent valid values concurrently.	Only ONE member can hold a valid value at any given instant.
Size Example	<code>sizeof(struct) = 4 + 1(pad 3) + 8 = 16 Bytes</code>	<code>sizeof(union) = max(4, 1, 8) = 8 Bytes</code>
Member Access	<code>var.id</code> (Direct) or <code>ptr->id</code> (Indirect via pointer)	<code>u.id</code> (Direct) or <code>u_ptr->id</code> (Indirect via pointer)

4.3 Dynamic Memory Allocation (DMA) Complete Lifecycle

Heap Management

FIGURE 4.1: DMA LIFECYCLE FLOWCHART



API Comparison: `malloc(sz)` allocates uninitialized raw bytes (contains garbage). `calloc(n, sz)` allocates memory and clears every bit to zero. `realloc(p, sz)` resizes an existing heap block, relocating and copying content if adjacent space is unavailable. `free(p)` deallocates memory back to the heap pool.

4.4 File Handling Operations & Stream Modes

Persistent Storage

Mode	Stream Meaning	File Missing Behavior	Existing File Behavior	Stream Functions
"r"	Read Text	Returns <code>NULL</code> error	Reads sequentially from byte 0	<code>fscanf()</code> , <code>fgetc()</code> , <code>fgets()</code>
"w"	Write Text	Creates new empty file	Truncates (erases) file to 0 bytes!	<code>fprintf()</code> , <code>fputc()</code> , <code>fputs()</code>
"a"	Append Text	Creates new empty file	Preserves contents; writes append at EOF	<code>fprintf()</code> , <code>fputs()</code>
"r+"	Read & Update	Returns <code>NULL</code> error	Reads & writes in-place without truncation	<code>fseek()</code> , <code>ftell()</code> , <code>rewind()</code>
"rb" / "wb"	Binary Stream	Standard open	No CR/LF character translation performed	<code>fread()</code> , <code>fwrite()</code>

UNIVERSITY EXAMINATION SOLVED QUESTIONS (MMDU PATTERN)

Comprehensive Solved Examination Questions Organized by Official MMDU Pattern • Diagrams Included in BOTH Question Statements AND Step-by-Step Solutions

SECTION A: CONCEPTUAL SHORT QUESTIONS (12 × 1 MARK)

Question 1 (Unit 1): Identify the missing intermediate file and translator in the C compilation sequence.

1 Mark

In the standard C compilation pipeline, complete the missing block [?] generated by the compiler `cc1` prior to machine code generation:

Question Schematic: `prog.c` → Preprocessor (`cpp`) → `prog.i` → Compiler (`cc1`) → [?] → Assembler (`as`) → `prog.o`.

STEP-BY-STEP SOLUTION:

Solution Schematic: `prog.i` (Expanded C) → Compiler (`cc1`) → Assembly Code (`prog.s`) → Assembler (`as`) → Relocatable Object (`prog.o`).

The compiler `cc1` translates expanded C code (`.i`) into architecture-specific **Assembly Code** (`prog.s`), which is subsequently assembled into machine object code (`prog.o`).

Question 2 (Unit 1): Identify the segment responsible for dynamically allocated memory in process RAM.

1 Mark

Refer to the process memory diagram below. Which labeled segment corresponds to memory allocated via `malloc()` ?

Question Schematic: [High Memory: Stack (↓)] → [Free Memory Space] → [Segment X (↑)] → [BSS] → [Initialized Data] → [Text].

STEP-BY-STEP SOLUTION:

Solution Schematic: Segment X is the **Heap Segment**, which expands upwards towards higher memory addresses as dynamic allocations occur.

Segment X = Heap. Heap memory is managed at runtime by functions in `<stdlib.h>` (`malloc` , `calloc` , `free`) and grows toward higher memory addresses.

Question 3 (Unit 1): Contrast the execution model of a Compiler versus an Interpreter.

1 Mark

Identify the fundamental architectural difference in how a compiler vs interpreter processes high-level code as depicted in the workflow below:

Question Schematic:

Path A: Source Code → [Whole-File Translation] → Object / Executable Binary → Fast Direct CPU Execution.

Path B: Source Code → [Line-by-Line Fetch/Decode/Execute Loop] → Output.

STEP-BY-STEP SOLUTION:

Solution Schematic: Path A = **Compiler** (Generates standalone machine code once). Path B = **Interpreter** (Translates instruction-by-instruction at runtime).

A **Compiler** translates the entire source program in a single pass into a relocatable binary executable, enabling rapid execution. An **Interpreter** interprets and executes source statements line-by-line without producing a permanent object file.

Question 4 (Unit 2): Evaluate the expression `z = x + y * ++x` given `x = 2, y = 3`.**1 Mark**

Determine the evaluation order in the expression tree for: `int x=2, y=3; int z = x + y * ++x;`

Question Schematic: Binary Operator Tree with root `+` having left operand `x` and right subtree `y * (++x)`.

STEP-BY-STEP SOLUTION:**Solution Schematic:**

Step 1: Prefix increment `++x` evaluates first $\rightarrow$ `x` becomes `3`.

Step 2: Multiplication: `y * x = 3 * 3 = 9`.

Step 3: Addition: `x + 9 = 3 + 9 = 12` (or compiler sequence point behavior).

Prefix `++` has tier 2 unary precedence, incrementing `x` to 3. Multiplication executes next (tier 3: `3 * 3 = 9`), followed by addition yielding `12`.

Question 5 (Unit 2): Explain why `float r = 5 / 2;` assigns `2.0` instead of `2.5`.**1 Mark**

Identify the truncation occurrence in the expression below:

Question Schematic: Constant `5` (int) $\div$ Constant `2` (int) $\rightarrow$ [**Intermediate Truncation**] $\rightarrow$ Assignment to `float r`.

STEP-BY-STEP SOLUTION:

Solution Schematic: `int / int  $\rightarrow$  int (5/2 = 2)` $\rightarrow$ Implicit float promotion upon assignment yields `2.000000`. Fix: `(float)5 / 2  $\rightarrow$  2.5`.

Both operands are integer literals, causing integer division that truncates the fractional remainder. The resulting integer `2` is promoted to `2.000000` upon storage in `r`.

Question 6 (Unit 2): Illustrate the execution flow of a switch statement when `break` is omitted.**1 Mark**

What execution defect occurs when `break;` is omitted across consecutive switch cases?

Question Schematic: `switch(ch)  $\rightarrow$  case '1': action1();  $\rightarrow$  [ ? ]  $\rightarrow$  case '2': action2();`.

STEP-BY-STEP SOLUTION:

Solution Schematic: Missing `break;` $\rightarrow$ **Fall-Through** occurs, cascading directly into `action2()` unconditionally.

Switch Fall-Through occurs. Execution cascades into subsequent case blocks sequentially until encountering a `break;` statement or the block terminator `}`.

Question 7 (Unit 3): Compute the physical address of element `A[1][2]` in a row-major 2D array `int A[3][4]`.**1 Mark**

Given base address 1000 and `sizeof(int) = 4` bytes, determine the linear address for element `A[1][2]` in `A[3][4]`:

Question Schematic: Row 0: [0,0][0,1][0,2][0,3] (4 elems) | Row 1: [1,0][1,1] [1,2] $\rightarrow$ Compute byte offset from Base.

STEP-BY-STEP SOLUTION:

Solution Schematic: Offset = $(i \times C + j) \times \text{Size} = (1 \times 4 + 2) \times 4 = 6 \times 4 = 24$ bytes.
Physical Address = `1000 + 24 = 1024`.

In row-major order, element `A[1][2]` is preceded by 1 complete row of 4 elements plus 2 elements in row 1, totaling 6 elements: `1000 + (6  $\times$  4) = 1024`.

Question 8 (Unit 3): Differentiate between `static` local variables and `auto` local variables.**1 Mark**

Contrast the lifetime of an `auto` variable against a `static` local variable:

Question Schematic: Function Call → Variable Created → Function Return → Variable Destroyed (Auto) vs Retained in RAM (Static).

STEP-BY-STEP SOLUTION:

Solution Schematic: `auto` lives on Stack (destroyed on return). `static` lives in Data Segment (retains value across calls for program duration).

An `auto` variable is created on the stack upon entry and destroyed upon function exit. A `static` variable persists in the Data Segment for the entire program execution lifetime.

Question 9 (Unit 3): What fatal error occurs if a recursive function lacks a base termination condition?**1 Mark**

Identify the runtime system failure when recursion continues indefinitely:

Question Schematic: Stack Segment: [Frame 1] → [Frame 2] → [Frame 3] → ... → [Stack Boundary Limit Exceeded].

STEP-BY-STEP SOLUTION:

Solution Schematic: Unbounded stack frame pushes deplete allocated stack memory → **Stack Overflow (SIGSEGV)** crash.

The process suffers a **Stack Overflow** error. Infinite stack activation frame allocations exhaust the virtual memory allocated for the call stack, crashing the operating system thread.

Question 10 (Unit 4): What is a Dangling Pointer, and how is it remediated?**1 Mark**

State the hazard of pointer `p` after invoking `free(p);`:

Question Schematic: Pointer `p` → [Address 0x5000] → Heap Block Released → Pointer `p` still retains `0x5000`.

STEP-BY-STEP SOLUTION:

Solution Schematic: Setting `p = NULL;` immediately after `free(p);` neutralizes the dangling pointer.

A **Dangling Pointer** references deallocated memory. Dereferencing it yields unpredictable behavior or memory corruption. Remediation: explicitly assign `p = NULL;` after deallocation.

Question 11 (Unit 4): Compare memory allocation for `struct` versus `union` with members `int x` and `char c`.**1 Mark**

Compute the respective storage requirements for `struct { int x; char c; }` vs `union { int x; char c; }`:

Question Schematic: Struct = [int (4B)] + [char (1B) + padding (3B)] vs Union = [Shared Overlay (4B)].

STEP-BY-STEP SOLUTION:

Solution Schematic: Struct size = `4 + 1 + 3 (pad) = 8 Bytes`. Union size = `max(4, 1) = 4 Bytes`.

The struct allocates sequential slots totaling **8 bytes** (with alignment padding). The union overlays members at address 0x0, requiring only **4 bytes** (size of largest member).

Question 12 (Unit 4): Define a Near Pointer in 16-bit 8086 segmented memory architecture.**1 Mark**

State the address bit-width and range limitations of a Near Pointer:

Question Schematic: Segment Register (DS) [Fixed 64KB Segment] → Near Pointer [16-bit Offset only].

STEP-BY-STEP SOLUTION:

Solution Schematic: Near Pointer = **16-bit (2 Bytes)** offset. Range = $2^{16} = 64$ KB within current segment.

A **Near Pointer** is a 16-bit address holding only the offset relative to the current default data segment register, restricting direct access to a **64 KB** boundary.

SECTION B: INTERMEDIATE ANALYTICAL QUESTIONS (4 × 2 MARKS)

Question 13 (Unit 1): Construct a dry-run execution trace table for finding the sum of digits of `N = 345`.

2 Marks

Trace the variable states during execution of the digit-sum loop below:

Question Flowchart: `while(n > 0) → rem = n % 10 → sum += rem → n /= 10 → Repeat until n == 0.`

STEP-BY-STEP SOLUTION:

Solution Trace Table:

Iteration	Initial <code>n</code>	Condition <code>n > 0</code>	<code>rem = n % 10</code>	<code>sum = sum + rem</code>	Updated <code>n = n / 10</code>
Init	345	-	-	0	345
Pass 1	345	True	5	$0 + 5 = 5$	34
Pass 2	34	True	4	$5 + 4 = 9$	3
Pass 3	3	True	3	$9 + 3 = 12$	0
Exit	0	False (Loop terminates)	-	12 (Final Sum)	0

The loop executes 3 iterations corresponding to the 3 decimal digits, accumulating digits 5, 4, and 3 to yield a final sum of 12.

Question 14 (Unit 2): Analyze short-circuit evaluation in `int a=0, b=5; if (a && ++b) ...`.

2 Marks

What are the final values of `a` and `b` after executing `if (a && ++b) ( )`?

Question Schematic: Logical AND Expression: Left Operand (`a`) && Right Operand (`++b`).

STEP-BY-STEP SOLUTION:

Solution Schematic: `a == 0` (False) → Short-Circuit Triggered → Right expression `++b` is completely bypassed and never evaluated!

Under ANSI C rules for logical `&&`, if the left operand evaluates to 0 (false), the result is guaranteed false. The compiler bypasses evaluation of the right operand. Thus, `++b` is never executed, and `b` remains 5. Result: `a = 0, b = 5`.

Question 15 (Unit 3): Differentiate Call by Value and Call by Reference using an integer swap function.

2 Marks

Illustrate memory stack behavior when swapping two integers via Call by Value vs Call by Reference:

Question Schematic: `main()` Frame: `[x=10, y=20]` → `swap()` Frame: `[a, b]`. Does mutating `a` and `b` modify `x` and `y`?

STEP-BY-STEP SOLUTION:

Solution Schematic:

Call by Value: `swap(x, y)` copies 10 and 20 into local params `a` and `b`. Local swap has zero effect on `x` and `y`.

Call by Reference: `swap(&x, &y)` passes memory addresses `0x100` and `0x104`. Dereferencing `*p` writes directly into `main()` stack memory, successfully swapping values.

```
// Call by Reference Swap:
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
} // Called via: swap(&x, &y);
```

Question 16 (Unit 4): Outline the Dynamic Memory Allocation lifecycle for an array of N integers.**2 Marks**

Write the ANSI C DMA lifecycle steps from initial allocation to safe deallocation:

Question Pipeline: Allocation → Validation Check → Usage → Deallocation → Nullification.

STEP-BY-STEP SOLUTION:**Solution Lifecycle Pipeline:**

`malloc(N * sizeof(int))` → `if (ptr == NULL) error()` → Process Data → `free(ptr)` → `ptr = NULL`.

```
int *arr = (int*)malloc(n * sizeof(int)); // 1. Allocate Heap
if (arr == NULL) { printf("Heap Exhausted!"); exit(1); } // 2. Guard Check
for(int i=0; i<n; i++) arr[i] = i * 10; // 3. Use Memory
free(arr); // 4. Free Heap Block
arr = NULL; // 5. Prevent Dangling Pointer
```

SECTION C: IN-DEPTH DERIVATION & ALGORITHMIC PROBLEMS (4 × 4 MARKS)**Question 17 (Unit 1): Design a Top-Down Flowchart and Algorithm to find all roots of a quadratic equation.****4 Marks**

Construct an algorithm and ANSI flowchart for finding the real and complex roots of $ax^2 + bx + c = 0$.

Question Conceptual Diagram: Input coefficients (a, b, c) → Compute Discriminant $D = b^2 - 4ac$ → 3-Way Branch: $D > 0$ (Real Distinct), $D = 0$ (Real Equal), $D < 0$ (Complex Conjugate).

STEP-BY-STEP SOLUTION:**Solution Flowchart Architecture:**

[Start] → Input `a, b, c` → `D = b*b - 4*a*c` → Decision Diamond: `D >= 0?`

→ **If True:** Sub-check `D == 0?` → If Equal: `r1 = r2 = -b/(2*a)`. If Distinct: `r1 = (-b + sqrt(D))/(2*a)`, `r2 = (-b - sqrt(D))/(2*a)`.

→ **If False:** `real = -b/(2*a)`, `imag = sqrt(-D/(2*a))` → Display `real ± i*imag` → [End].

```
#include <stdio.h>
#include <math.h>
void solveQuadratic(double a, double b, double c) {
    if (a == 0) { printf("Not a quadratic equation!\n"); return; }
    double d = b*b - 4*a*c;
    if (d > 0) {
        double r1 = (-b + sqrt(d)) / (2*a), r2 = (-b - sqrt(d)) / (2*a);
        printf("Real & Distinct Roots: %.2f, %.2f\n", r1, r2);
    } else if (d == 0) {
        printf("Real & Equal Roots: %.2f, %.2f\n", -b / (2*a), -b / (2*a));
    } else {
        double real = -b / (2*a), imag = sqrt(-d/(2*a));
        printf("Complex Roots: %.2f + %.2fi, %.2f - %.2fi\n", real, imag, real, imag);
    }
}
```

Question 18 (Unit 2): Implement Matrix Multiplication using nested loops with algorithmic trace.**4 Marks**

Explain the 3-tier nested loop control structure required for multiplying matrices $A_{M \times K}$ and $B_{K \times N}$.

Question Schematic: Row i of Matrix A • Column j of Matrix B = Element $C[i][j] = \sum (A[i][k] \times B[k][j])$.

STEP-BY-STEP SOLUTION:**Solution 3-Tier Loop Nesting Model:**

Outer Loop `for(i=0; i<M; i++)` → Iterates across Rows of A.

Middle Loop `for(j=0; j<N; j++)` → Iterates across Columns of B.

Inner Accumulator Loop `for(k=0; k<K; k++)` → Computes dot product: `C[i][j] += A[i][k] * B[k][j]`.

```
void multiplyMatrices(int A[2][2], int B[2][2], int C[2][2]) {
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            C[i][j] = 0;
            for (int k = 0; k < 2; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

Complexity Analysis: The 3 nested loops execute $M \times N \times K$ times, yielding a computational time complexity of $O(M \times N \times K)$ (or $O(N^3)$ for square $N \times N$ matrices).

Question 19 (Unit 3): Develop Binary Search algorithm and prove its $O(\log N)$ complexity.**4 Marks**

Present the divide-and-conquer strategy of Binary Search with a visual search space halving trace.

Question Schematic: Sorted Array: [2, 5, 8, 12, 16, 23, 38, 56, 72, 91] (Target = 23). Search Range: `low` to `high`.

STEP-BY-STEP SOLUTION:**Solution Divide-and-Conquer Trace:**

Pass 1: `low=0, high=9` → `mid=(0+9)/2=4` → `a[4]=16 < 23` → Discard left half! New range: `low=5, high=9`.

Pass 2: `low=5, high=9` → `mid=(5+9)/2=7` → `a[7]=56 > 23` → Discard right half! New range: `low=5, high=6`.

Pass 3: `low=5, high=6` → `mid=(5+6)/2=5` → `a[5]=23 == 23` → **Target Found at Index 5 in 3 steps!**

```
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2; // Prevents integer overflow
        if (arr[mid] == key) return mid; // Found
        else if (arr[mid] < key) low = mid + 1; // Search right half
        else high = mid - 1; // Search left half
    }
    return -1; // Not found
}
```

Complexity Proof: Each comparison halves the remaining array size: $N \rightarrow N/2 \rightarrow N/4 \rightarrow \dots \rightarrow N/2^k = 1$. Solving $2^k = N$ yields $k = \log_2(N)$. Hence, maximum comparisons are $O(\log N)$.

Question 20 (Unit 4): Explain Near, Far, and Huge Pointers in 16-bit 8086 Real-Mode Architecture.

4 Marks

Detail the internal segmentation arithmetic and pointer comparison issues associated with Near, Far, and Huge pointers.

Question Architecture Schematic: 8086 Physical Address = (Segment × 16) + Offset (20-bit address bus, 1 MB total RAM).

STEP-BY-STEP SOLUTION:

Solution Segmentation Architecture:

Pointer Type	Size	Format	Memory Span	Arithmetic & Comparison Behavior
Near	16-bit (2 B)	Offset only	64 KB (single segment)	Fast arithmetic; cannot cross segment boundaries.
Far	32-bit (4 B)	Segment : Offset	1 MB (full address space)	Unnormalized. Pointers <code>0x1000:0x0010</code> and <code>0x1001:0x0000</code> point to the SAME byte, but <code>p1 == p2</code> evaluates FALSE!
Huge	32-bit (4 B)	Normalized Seg:Offset	1 MB (full address space)	Auto-normalizes offset ≤ 15 on every increment. Relational comparisons are mathematically valid across segments.

Huge Pointer Normalization: In a huge pointer, any offset ≥ 16 is immediately converted: Segment ← Segment + $\frac{\text{Offset}}{16}$ and Offset ← Offset % 16. This ensures a unique representation for each physical byte in RAM.

SECTION D: COMPREHENSIVE SYSTEM DESIGN & CODING PROBLEMS (4 × 6 MARKS)

Question 21 (Units 1 & 2): Comprehensive C Compilation Pipeline, Token Processing & Operator Precedence.

6 Marks

Trace the transformation of the C statement `int result = a + b * c > d && !flag;` through lexical tokenization, operator tree parsing, and intermediate code generation.

Question Pipeline: Source Text → Lexer (Tokens) → Parser (AST) → Semantic Type Checking → Code Generator.

STEP-BY-STEP SOLUTION:

Solution AST (Abstract Syntax Tree) Hierarchy:

Root: `&&` (Tier 7)
→ Left Subtree: `>` (Tier 5)
→ Left: `+` (Tier 4) → [`a`, `*` (Tier 3: `b * c`)]
→ Right: `d`
→ Right Subtree: `!` (Tier 2 Unary: `!flag`)

- Step 1: Lexical Analysis:** Stream of characters is tokenized into: `[KWINT, IDENT("result"), OPASSIGN, IDENT("a"), OPPLUS, IDENT("b"), OPMULT, IDENT("c"), OPGT, IDENT("d"), OPAND, OPNOT, IDENT("flag"), SEMICOLON]`.
- Step 2: Precedence-Driven Parsing:**
 - Unary `!flag` evaluated first (tier 2).
 - Multiplication `b * c` evaluated (tier 3).
 - Addition `a + (b * c)` evaluated (tier 4).
 - Relational comparison `(a + b*c) > d` evaluated (tier 5).
 - Logical AND short-circuiting applied between relational result and `!flag` (tier 7).

```
// Equivalent fully parenthesized semantic expression:  
int result = (((a + (b * c)) > d) && (!flag));
```

(a) Provide a complete trace of Bubble Sort sorting the array [64, 34, 25, 12, 22, 11, 90]. (b) Draw the call stack activation records during recursive computation of N! = 4!.

Question Schematic:

Part (a): Unsorted Array → Sinking passes 1 to N-1 → Fully sorted.

Part (b): Stack Frames for fact(4) → fact(3) → fact(2) → fact(1).

STEP-BY-STEP SOLUTION:

Part (a): Bubble Sort Complete Pass Trace:

Pass	Array State After Pass	Element Settled	Swaps Performed
Initial	[64, 34, 25, 12, 22, 11, 90]	-	-
Pass 1	[34, 25, 12, 22, 11, 64, 90]	90 settled at index 6	5 swaps
Pass 2	[25, 12, 22, 11, 34, 64, 90]	64 settled at index 5	4 swaps
Pass 3	[12, 22, 11, 25, 34, 64, 90]	34 settled at index 4	3 swaps
Pass 4	[12, 11, 22, 25, 34, 64, 90]	25 settled at index 3	1 swap
Pass 5	[11, 12, 22, 25, 34, 64, 90]	22 settled at index 2	1 swap
Pass 6	[11, 12, 22, 25, 34, 64, 90]	11, 12 sorted	0 swaps (Early exit)

Part (b): Factorial 4! Recursion Stack Unwinding Diagram:

- fact(4) pushed → calls 4 * fact(3)
- fact(3) pushed → calls 3 * fact(2)
- fact(2) pushed → calls 2 * fact(1)
- fact(1) pushed → hits base condition n <= 1, returns 1.

Unwinding Phase: Frame 2 gets 1 → returns 2 × 1 = 2. Frame 3 gets 2 → returns 3 × 2 = 6. Frame 4 gets 6 → returns 4 × 6 = 24.

```
long factorial(int n) {
    if (n <= 1) return 1; // Base condition
    return n * factorial(n - 1); // Recursive call
}
```

Question 23 (Unit 4): Student Records Management System using Dynamic Memory Allocation and File I/O.**6 Marks**

Design an architecture in ANSI C to dynamically allocate an array of **Student** structures on the heap, collect input from the user, save records to a binary file **students.dat**, and read them back.

Question System Architecture: RAM (Heap Struct Array) → **fwrite()** → Disk (**students.dat**) → **fread()** → RAM (Report Display).

STEP-BY-STEP SOLUTION:**Solution Memory & Stream Bridge Model:**

Student* students = malloc(N * sizeof(Student)) → Heap Allocation → Binary File Stream **FILE *fp = fopen("students.dat", "wb")** → Block persistence via **fwrite(students, sizeof(Student), N, fp)**.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int rollNo;
    char name[32];
    float gpa;
} Student;

void processRecords(int n) {
    // 1. Dynamic Allocation
    Student *list = (Student*)malloc(n * sizeof(Student));
    if (!list) { printf("Memory allocation failed!\n"); return; }

    // 2. Initialize sample record
    list[0].rollNo = 101; sprintf(list[0].name, "Aman"); list[0].gpa = 8.85f;

    // 3. Persist to Binary File
    FILE *fp = fopen("students.dat", "wb");
    if (fp) {
        fwrite(list, sizeof(Student), n, fp);
        fclose(fp);
    }

    // 4. Read back into fresh buffer
    Student *readList = (Student*)malloc(n * sizeof(Student));
    fp = fopen("students.dat", "rb");
    if (fp) {
        fread(readList, sizeof(Student), n, fp);
        printf("Recovered Record: Roll %d, Name: %s, GPA: %.2f\n",
            readList[0].rollNo, readList[0].name, readList[0].gpa);
        fclose(fp);
    }

    // 5. Clean up heap memory
    free(list); free(readList);
}
```

Question 24 (Unit 4): Callback Architecture and Generic Sorting using Function Pointers.**6 Marks**

Explain the mechanism of Function Pointers in C by implementing a generic bubble sort that accepts a comparison callback function (supporting both ascending and descending sorts).

Question Function Pointer Schematic: `bubbleSort(arr, n, compareCallback)` → Invokes `compareCallback(a, b)` located in Text Segment.

STEP-BY-STEP SOLUTION:**Solution Callback Execution Model:**

Function pointer `int (*cmp)(int, int)` holds the starting address of either `ascending()` or `descending()` in the code/text segment. Passing this pointer decouples sorting logic from data ordering policies.

```
#include <stdio.h>

// Comparison callbacks:
int ascending(int a, int b) { return a > b; }
int descending(int a, int b) { return a < b; }

// Generic Bubble Sort accepting a Function Pointer:
void genericSort(int arr[], int n, int (*compare)(int, int)) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (compare(arr[j], arr[j + 1])) { // Dynamic callback!
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int data[] = {45, 12, 89, 34};
    genericSort(data, 4, ascending); // Sorts [12, 34, 45, 89]
    genericSort(data, 4, descending); // Sorts [89, 45, 34, 12]
    return 0;
}
```

Architectural Significance: Function pointers establish high-order modularity in C, allowing runtime polymorphism and clean event-driven architectures without needing object-oriented inheritance.